

Principal Component Analysis

From Linear Algebra Foundations to Applications

1 Linear Algebra Prerequisites

1.1 Matrices and Notation

A matrix A of size $n \times m$ has n rows and m columns. We write a_{ij} for the element in row i , column j .

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{pmatrix}_{n \times m}$$

A **square matrix** has $n = m$. It is **symmetric** if $A = A^\top$, i.e. $a_{ij} = a_{ji}$.

1.2 Orthogonal Projections

Definition: Orthogonal Projection

The projection of vector $\mathbf{x} \in \mathbb{R}^d$ onto a unit vector $\mathbf{u}$ (i.e. $\|\mathbf{u}\| = 1$) is:

$$\text{proj}_{\mathbf{u}}(\mathbf{x}) = (\mathbf{u}^\top \mathbf{x}) \mathbf{u}$$

The **scalar coordinate** along $\mathbf{u}$ is the dot product $\mathbf{u}^\top \mathbf{x}$.

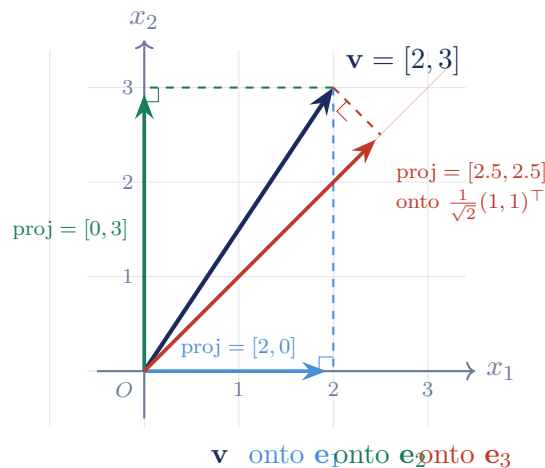


Figure 1: Projections of $\mathbf{v} = [2, 3]^\top$ onto three unit vectors: $\mathbf{e}_1 = [1, 0]^\top$ (blue), $\mathbf{e}_2 = [0, 1]^\top$ (green), and $\mathbf{e}_3 = \frac{1}{\sqrt{2}}[1, 1]^\top$ (red). Dashed lines show the perpendicular drop; the coloured vectors are the orthogonal projections $(\hat{\mathbf{u}}^\top \mathbf{v})\hat{\mathbf{u}}$.

1.3 Eigenvalues and Eigenvectors

Definition: Eigenvector / Eigenvalue

For a square matrix $A \in \mathbb{R}^{m \times m}$, a non-zero vector $\mathbf{v} \in \mathbb{R}^m$ and scalar $\lambda \in \mathbb{R}$ satisfying

$$A\mathbf{v} = \lambda\mathbf{v}$$

are called an **eigenvector** and **eigenvalue** of A , respectively.

[!] Intuition

Most vectors, when multiplied by a matrix, change both their length *and* direction. Eigenvectors are special: multiplying by A only **stretches or flips** them—their direction is preserved. The eigenvalue λ tells you how much stretching happens.

Example: A 2×2 Example

Let $A = \begin{pmatrix} 3 & 1 \\ 0 & 2 \end{pmatrix}$. Try $\mathbf{v} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$:

$$A\mathbf{v} = \begin{pmatrix} 3 \\ 0 \end{pmatrix} = 3\mathbf{v} \quad \Rightarrow \quad \lambda = 3, \mathbf{v} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

The vector pointing in the x -direction is simply scaled by 3. Now try $\mathbf{v} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$:

$$A\mathbf{v} = \begin{pmatrix} 2 \\ -2 \end{pmatrix} = 2\mathbf{v} \quad \Rightarrow \quad \lambda = 2, \mathbf{v} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

This matrix has two eigenvectors; together they describe all of its “stretching” behaviour.

1.3.1 The Determinant

Before we can compute eigenvalues, we need one more tool: the **determinant**.

Definition: Determinant

The determinant of a square matrix $A \in \mathbb{R}^{n \times n}$, written $\det(A)$ or $|A|$, is a single number that encodes key properties of the matrix. It is defined by:

- 2×2 : $\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$.
- 3×3 (**cofactor expansion along row 1**):

$$\det \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} = a \det \begin{pmatrix} e & f \\ h & i \end{pmatrix} - b \det \begin{pmatrix} d & f \\ g & i \end{pmatrix} + c \det \begin{pmatrix} d & e \\ g & h \end{pmatrix}$$

Each 2×2 sub-determinant (called a **minor**) is obtained by deleting the row and column of the multiplying entry. The signs alternate $+, -, +$.

- **Larger matrices:** apply cofactor expansion recursively—each $n \times n$ determinant reduces to n determinants of size $(n-1) \times (n-1)$.

The key property: $\det(A) = 0$ if and only if A is **singular** (non-invertible). This is exactly the condition we need to find eigenvalues.

Example: Computing Determinants

2×2 :

$$\det \begin{pmatrix} 3 & 1 \\ 1 & 2 \end{pmatrix} = 3 \cdot 2 - 1 \cdot 1 = 5 \neq 0 \quad (\text{invertible})$$

$$\det \begin{pmatrix} 2 & 4 \\ 1 & 2 \end{pmatrix} = 2 \cdot 2 - 4 \cdot 1 = 0 \quad (\text{singular: row 2 is half of row 1})$$

3×3 (**step by step**): Let $B = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 1 & 0 & 6 \end{pmatrix}$.

Expand along row 1:

$$\begin{aligned} \det(B) &= 1 \cdot \det \begin{pmatrix} 4 & 5 \\ 0 & 6 \end{pmatrix} - 2 \cdot \det \begin{pmatrix} 0 & 5 \\ 1 & 6 \end{pmatrix} + 3 \cdot \det \begin{pmatrix} 0 & 4 \\ 1 & 0 \end{pmatrix} \\ &= 1 \cdot (4 \cdot 6 - 5 \cdot 0) - 2 \cdot (0 \cdot 6 - 5 \cdot 1) + 3 \cdot (0 \cdot 0 - 4 \cdot 1) \\ &= 1 \cdot 24 - 2 \cdot (-5) + 3 \cdot (-4) \\ &= 24 + 10 - 12 = 22 \neq 0 \quad (\text{invertible}) \end{aligned}$$

1.3.2 Computing Eigenvalues and Eigenvectors

Starting from $A\mathbf{v} = \lambda\mathbf{v}$:

$$A\mathbf{v} - \lambda I\mathbf{v} = \mathbf{0} \quad \Rightarrow \quad (A - \lambda I)\mathbf{v} = \mathbf{0}$$

For non-zero $\mathbf{v}$, the matrix $(A - \lambda I)$ must be *singular* (i.e. have determinant zero):

$$\boxed{\det(A - \lambda I) = 0} \quad (\text{characteristic equation — solve for } \lambda)$$

Once λ is found, substitute back into $A\mathbf{v} = \lambda\mathbf{v}$ and solve for $\mathbf{v}$.

Example: Worked Characteristic Equation

For $A = \begin{pmatrix} 4 & 2 \\ 1 & 3 \end{pmatrix}$:

$$\det(A - \lambda I) = (4 - \lambda)(3 - \lambda) - 2 = \lambda^2 - 7\lambda + 10 = 0$$

$$\lambda_1 = 5, \quad \lambda_2 = 2$$

For $\lambda_1 = 5$: $(A - 5I)\mathbf{v} = \mathbf{0}$ gives $\mathbf{v}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$ (normalise: divide by $\sqrt{5}$).

For $\lambda_2 = 2$: $(A - 2I)\mathbf{v} = \mathbf{0}$ gives $\mathbf{v}_2 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$ (normalise: divide by $\sqrt{2}$).

1.4 Positive Semi-Definite (PSD) Matrices

Definition: Positive Semi-Definite Matrix

A symmetric matrix $A \in \mathbb{R}^{d \times d}$ is **positive semi-definite (PSD)**, written $A \succeq 0$, if

$$x^\top A x \geq 0 \quad \forall x \in \mathbb{R}^d.$$

It is **positive definite (PD)**, written $A \succ 0$, if the inequality is strict for all $x \neq 0$.

1.5 The Spectral Theorem

Theorem 1.1 (Spectral Theorem for Real Symmetric Matrices). *Every real symmetric matrix $A \in \mathbb{R}^{d \times d}$ ($A = A^\top$) satisfies:*

- (i) **All eigenvalues are real:** $\lambda_1, \dots, \lambda_d \in \mathbb{R}$.
- (ii) **Eigenvectors are orthogonal:** Eigenvectors for distinct eigenvalues are orthogonal; repeated eigenvalues admit an orthonormal basis.
- (iii) **A is orthogonally diagonalisable:**

$$A = Q \Lambda Q^\top$$

where $Q = [q_1 \mid \dots \mid q_d]$ is orthogonal ($Q^\top Q = I$) with columns being orthonormal eigenvectors, and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$.

This decomposition is also called the **eigendecomposition** of A , and it is the mathematical engine that powers PCA.

1.6 Covariance Matrices

Definition: Covariance Matrix

Given n samples $x^{(1)}, \dots, x^{(n)} \in \mathbb{R}^d$, the **covariance matrix** is

$$\mathbf{S} = \frac{1}{n} \sum_{j=1}^n (x^{(j)} - \bar{x})(x^{(j)} - \bar{x})^\top, \quad \bar{x} = \frac{1}{n} \sum_{j=1}^n x^{(j)}.$$

Writing $\tilde{X} \in \mathbb{R}^{d \times n}$ for the mean-centred data matrix (whose j -th column is $x^{(j)} - \bar{x}$):

$$\mathbf{S} = \frac{1}{n} \tilde{X} \tilde{X}^\top.$$

[!] Intuition

The (i, j) entry of $\mathbf{S}$ measures how features i and j *co-vary*. If tall people tend to be heavier (features positively correlated), that entry will be positive. Diagonal entries $\mathbf{S}_{ii}$ are simply the variance of feature i . A large diagonal entry means that feature is “spread out”; PCA will favour directions aligned with large entries.

Example: Covariance Matrix for 2 Features

Suppose you have student data: feature 1 = hours studied, feature 2 = exam score. If students who study more tend to score higher, then $\mathbf{S}_{12} > 0$ (positive covariance). If everyone scores the same regardless of study time, $\mathbf{S}_{12} = 0$. PCA uses these relationships to find the most informative projection direction.

Worked Example: Covariance of a Small Dataset

Consider $n = 4$ samples in $\mathbb{R}^2$:

$$x^{(1)} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad x^{(2)} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}, \quad x^{(3)} = \begin{pmatrix} 5 \\ 4 \end{pmatrix}, \quad x^{(4)} = \begin{pmatrix} 3 \\ 6 \end{pmatrix}.$$

Step 1: Compute the sample mean.

$$\bar{x} = \frac{1}{4} \left(\begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 3 \\ 4 \end{pmatrix} + \begin{pmatrix} 5 \\ 4 \end{pmatrix} + \begin{pmatrix} 3 \\ 6 \end{pmatrix} \right) = \frac{1}{4} \begin{pmatrix} 12 \\ 16 \end{pmatrix} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}.$$

Step 2: Mean-centre the data.

$$x^{(1)} - \bar{x} = \begin{pmatrix} -2 \\ -2 \end{pmatrix}, \quad x^{(2)} - \bar{x} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad x^{(3)} - \bar{x} = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \quad x^{(4)} - \bar{x} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}.$$

The mean-centred data matrix is

$$\tilde{X} = \begin{pmatrix} -2 & 0 & 2 & 0 \\ -2 & 0 & 0 & 2 \end{pmatrix} \in \mathbb{R}^{2 \times 4}.$$

Step 3: Compute $\mathbf{S} = \frac{1}{n} \tilde{X} \tilde{X}^\top$.

$$\tilde{X} \tilde{X}^\top = \begin{pmatrix} -2 & 0 & 2 & 0 \\ -2 & 0 & 0 & 2 \end{pmatrix} \begin{pmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 0 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 8 & 4 \\ 4 & 8 \end{pmatrix}.$$

Therefore, with $n = 4$:

$$\mathbf{S} = \frac{1}{4} \begin{pmatrix} 8 & 4 \\ 4 & 8 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

The Covariance Matrix is Always PSD

Notice in the example above that both eigenvalues of $\mathbf{S}$ are non-negative. This is not a coincidence — it holds for any covariance matrix, for any dataset.

Theorem 1.2. *The covariance matrix $\mathbf{S} = \frac{1}{n} \tilde{X} \tilde{X}^\top$ is positive semi-definite, i.e. $\mathbf{S} \succeq 0$.*

Proof. For any vector $z \in \mathbb{R}^d$:

$$z^\top \mathbf{S} z = z^\top \left(\frac{1}{n} \tilde{X} \tilde{X}^\top \right) z = \frac{1}{n} z^\top \tilde{X} \tilde{X}^\top z = \frac{1}{n} \|\tilde{X}^\top z\|^2 \geq 0.$$

The last step holds because $\|\cdot\|^2 \geq 0$ for any vector. □

Key Result

Since $\mathbf{S} \succeq 0$, all eigenvalues of the covariance matrix are non-negative:
 $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0$.

2 What is PCA Useful For?

2.1 The Curse of Dimensionality

Modern data is almost always *high-dimensional*. Consider a few examples:

Dataset	Samples (n)	Raw features (d)	Useful dims?
100 × 100 grayscale image	1 image	10 000 pixels	$\ll 100$
Gene expression data	500 patients	20 000 genes	few hundred
Text (bag-of-words)	10^6 docs	10^5 words	a few thousand
Finance	1 year of data	500 stock prices	< 20 factors

High dimensions cause real problems:

- **Computational cost:** Algorithms scale as $O(d^2)$ or worse.
- **Statistical inefficiency:** With many features, you need exponentially more data to cover the space.
- **Visualization:** Humans can only visualise 2D/3D.
- **Redundancy:** Many features are correlated; they carry the same information twice.

[!] Intuition

Think of a photograph of a face. It has 10,000+ pixel values, but the *information* needed to recognise it—gender, age, expression—lives in a much smaller space. PCA finds that smaller space automatically.

2.2 Application: Data Visualisation

High-dimensional data is invisible to human eyes. PCA projects it to 2D or 3D for exploration.

Example: MNIST Digit Recognition

Each handwritten digit image is $28 \times 28 = 784$ pixels, giving $d = 784$ features. Using PCA to project to 2D:

- Digits of the same class (same number) cluster together in 2D.
- Similar-looking digits (e.g. 3 and 8) appear close in 2D space.
- You can visually confirm whether classes are linearly separable before running any classifier.

All this from just 2 numbers per image instead of 784!

3 What is PCA? The Core Idea

Definition: Principal Component Analysis (PCA)

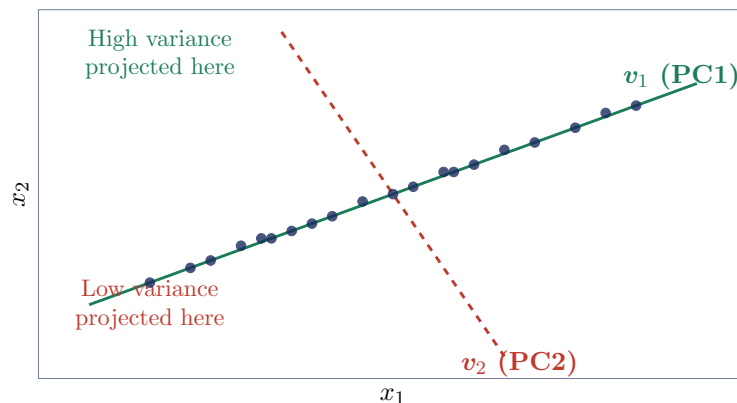
PCA is a technique that transforms data from a high-dimensional space ($\mathbb{R}^d$) into a low-dimensional space ($\mathbb{R}^p$, $p < d$) by finding a set of **orthogonal axes** (the *principal components*) along which the data varies the most.

[!] Intuition

Imagine you're photographing a long thin rod floating in 3D space. If you photograph it from the end, it looks like a dot—you lose all information about its length. If you photograph it from the side, you see its full extent. PCA automatically finds the “side view”—the direction that shows the most variation.

3.1 Geometric Picture

Consider n data points in $\mathbb{R}^2$. We want to project them onto a 1D line through the origin.



Key observation: v_1 is a better choice because projecting data onto it preserves more *spread* (variance). Projections onto v_2 (which is *perpendicular* to v_1) squash all points together, losing information. The two principal components are always orthogonal to each other.

3.2 Formalising the Objective

Setup: We have n data points $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$. We want a unit vector $\mathbf{u}_1 \in \mathbb{R}^d$ that maximises the variance of the projected scalar coordinates $z_i = \mathbf{u}_1^\top \mathbf{x}_i$.

3.2.1 Step 1 — Mean of projections

The mean of all projected points is:

$$\bar{z} = \frac{1}{n} \sum_{i=1}^n \mathbf{u}_1^\top \mathbf{x}_i = \mathbf{u}_1^\top \bar{\mathbf{x}}$$

3.2.2 Step 2 — Variance of projections

$$\begin{aligned} \text{Var}(z) &= \frac{1}{n} \sum_{i=1}^n (z_i - \bar{z})^2 = \frac{1}{n} \sum_{i=1}^n \mathbf{u}_1^\top (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top \mathbf{u}_1 \\ &= \mathbf{u}_1^\top \left[\frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top \right] \mathbf{u}_1 = \mathbf{u}_1^\top \mathbf{S} \mathbf{u}_1 \end{aligned}$$

where $\mathbf{S} = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top$ is the **covariance matrix** (as defined in Section 1).

3.2.3 Step 3 — Optimisation problem

PCA finds the first principal component $\mathbf{u}_1$ by solving:

$$\max_{\mathbf{u}_1} \mathbf{u}_1^\top \mathbf{S} \mathbf{u}_1 \quad \text{subject to} \quad \mathbf{u}_1^\top \mathbf{u}_1 = 1$$

[!] Watch Out

Is this a convex problem? The objective $f(\mathbf{u}_1) = \mathbf{u}_1^\top \mathbf{S} \mathbf{u}_1$ is convex (since $\nabla^2 f = 2\mathbf{S} \succeq 0$). But we are *maximising* a convex function over a non-convex constraint set (the unit sphere). This is not a standard convex program—we need to use a clever algebraic trick instead.

3.3 Solving the Optimisation via the Spectral Theorem

Since $\mathbf{S}$ is real and symmetric, the Spectral Theorem (Section 1.4) guarantees d orthonormal eigenvectors $\mathbf{v}_1, \dots, \mathbf{v}_d$ with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0$.

We can write:

$$\mathbf{S} = \mathbf{V} \mathbf{\Sigma} \mathbf{V}^\top$$

where $\mathbf{V} = [\mathbf{v}_1 \mid \mathbf{v}_2 \mid \dots \mid \mathbf{v}_d]$ and $\mathbf{\Sigma} = \text{diag}(\lambda_1, \dots, \lambda_d)$, with $\mathbf{V}^\top \mathbf{V} = \mathbf{I}$.

Any unit vector $\mathbf{u}_1$ can be written as $\mathbf{u}_1 = \mathbf{V} \boldsymbol{\alpha}^\top$ with $\sum_{i=1}^d \alpha_i^2 = 1$. Substituting:

$$\mathbf{u}_1^\top \mathbf{S} \mathbf{u}_1 = \boldsymbol{\alpha} \mathbf{\Sigma} \boldsymbol{\alpha}^\top = \sum_{i=1}^d \alpha_i^2 \lambda_i$$

Key Result

The objective $\sum_i \alpha_i^2 \lambda_i$ is a weighted average of λ_i 's with weights α_i^2 summing to 1. The maximum is achieved by putting all weight on the largest eigenvalue:

$$\mathbf{u}_1 = \mathbf{v}_1 \quad \text{where} \quad \lambda_1 = \max(\lambda_1, \dots, \lambda_d)$$

The first principal component is the eigenvector of $\mathbf{S}$ with the largest eigenvalue.

3.4 Finding More Components

After finding $\mathbf{u}_1$, find $\mathbf{u}_2$ by solving the same problem with one extra constraint: $\mathbf{u}_2 \perp \mathbf{u}_1$:

$$\max_{\mathbf{u}_2} \mathbf{u}_2^\top \mathbf{S} \mathbf{u}_2 \quad \text{s.t.} \quad \|\mathbf{u}_2\| = 1, \quad \mathbf{u}_1^\top \mathbf{u}_2 = 0$$

$\Rightarrow \mathbf{u}_2 = \text{eigenvector of } \mathbf{S} \text{ with second largest eigenvalue}$

Key Result

The k -th principal component $\mathbf{u}_k$ is the eigenvector of $\mathbf{S}$ corresponding to the k -th largest eigenvalue λ_k . The top p eigenvectors $\{\mathbf{u}_1, \dots, \mathbf{u}_p\}$ define the best p -dimensional subspace for the data.

4 The PCA Algorithm

PCA Algorithm

Input: n samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$; target dimension $p < d$.

1. Compute the mean:

$$\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$$

2. Mean-centre the data:

$$\tilde{\mathbf{x}}_i = \mathbf{x}_i - \bar{\mathbf{x}} \quad \forall i = 1, \dots, n$$

3. Compute the covariance matrix:

$$\mathbf{S} = \frac{1}{n} \sum_{i=1}^n \tilde{\mathbf{x}}_i \tilde{\mathbf{x}}_i^\top \in \mathbb{R}^{d \times d}$$

4. Eigenvalue decomposition:

$$\mathbf{S} = \mathbf{V} \mathbf{\Sigma} \mathbf{V}^\top$$

with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0$.

5. Pick the top p eigenvectors:

$$\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_p \quad (\text{principal components})$$

6. Form the projection matrix:

$$\mathbf{U} = [\mathbf{u}_1 \mid \mathbf{u}_2 \mid \dots \mid \mathbf{u}_p] \in \mathbb{R}^{d \times p}$$

7. Project each data point:

$$\mathbf{y}_i = \mathbf{U}^\top \tilde{\mathbf{x}}_i \in \mathbb{R}^p$$

Output: Reduced representations $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^p$.

4.1 Why Mean-Centre? (Step 2)

4.2 Why Normalise? (Before Step 1)

4.3 Choosing p : How Many Components?

The eigenvalue λ_k tells you the **variance explained** by the k -th principal component. The fraction of total variance explained by the top p components is:

$$\text{Explained variance ratio} = \frac{\sum_{k=1}^p \lambda_k}{\sum_{k=1}^d \lambda_k}$$

A common heuristic: choose p so that $> 90\%$ (or $> 95\%$) of variance is explained.

Principal Component Analysis

1 Full Worked Example: 2D $\rightarrow$ 1D

Example: Step-by-Step PCA on Toy Data

Data: Four points in $\mathbb{R}^2$:

$$\mathbf{x}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad \mathbf{x}_2 = \begin{pmatrix} 4 \\ 3 \end{pmatrix}, \quad \mathbf{x}_3 = \begin{pmatrix} 6 \\ 5 \end{pmatrix}, \quad \mathbf{x}_4 = \begin{pmatrix} 8 \\ 7 \end{pmatrix}$$

Step 1 — Mean: $\bar{\mathbf{x}} = \frac{1}{4}(2 + 4 + 6 + 8, 1 + 3 + 5 + 7)^\top = (5, 4)^\top$

Step 2 — Centre: $\tilde{\mathbf{x}}_1 = (-3, -3)^\top$, $\tilde{\mathbf{x}}_2 = (-1, -1)^\top$, $\tilde{\mathbf{x}}_3 = (1, 1)^\top$, $\tilde{\mathbf{x}}_4 = (3, 3)^\top$

Step 3 — Covariance:

$$\mathbf{S} = \frac{1}{n} [(-3, -3)^\top (-3, -3) + \dots] = \frac{1}{4} \begin{pmatrix} 20 & 20 \\ 20 & 20 \end{pmatrix} = \begin{pmatrix} 5 & 5 \\ 5 & 5 \end{pmatrix}$$

Step 4 — Eigendecomposition: $\det(\mathbf{S} - \lambda \mathbf{I}) = (5 - \lambda)^2 - 25 = 0$

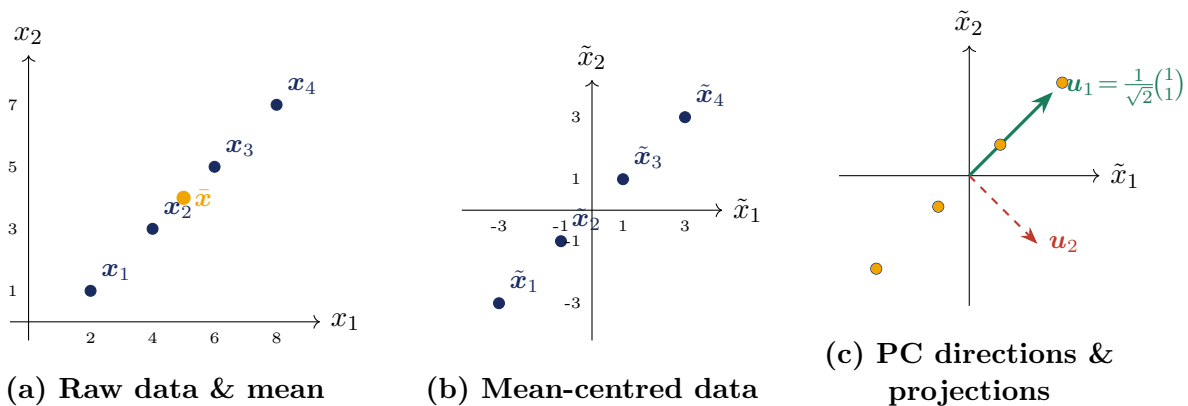
$$\lambda_1 = 10, \quad \lambda_2 = 0 \quad \Rightarrow \quad \mathbf{u}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \mathbf{u}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Steps 5–7 — Project: With $\mathbf{u}_1 = \frac{1}{\sqrt{2}}(1, 1)^\top$:

$$y_i = \mathbf{u}_1^\top \tilde{\mathbf{x}}_i : \quad -3\sqrt{2}, \quad -\sqrt{2}, \quad \sqrt{2}, \quad 3\sqrt{2}$$

Interpretation: All variance is along the $(1, 1)$ direction (points lie on a line!). PC2 has $\lambda_2 = 0$: zero variance. We can *perfectly* represent this data in 1D with zero information loss.

Visualising the worked example. The three panels below trace the geometry of the computation.



In panel (c) the dashed lines drop each point perpendicularly onto $\mathbf{u}_1$. Because all four centred points lie exactly on the $\tilde{x}_1 = \tilde{x}_2$ diagonal, every point *already* lives on PC1—the projections (amber dots) coincide with the original points, confirming that $\lambda_2 = 0$ and no information is lost.

2 Extension: 3D $\rightarrow$ 2D Example

Example: PCA: $\mathbb{R}^3 \rightarrow \mathbb{R}^2$

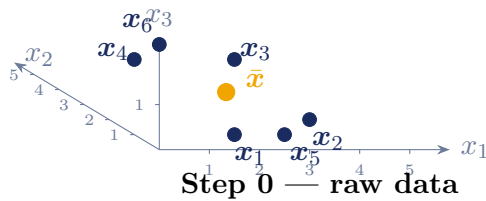
Data: Six points in $\mathbb{R}^3$ that lie roughly on a tilted plane:

$$\mathbf{x}_1 = \begin{pmatrix} 2 \\ 1 \\ 0 \end{pmatrix}, \mathbf{x}_2 = \begin{pmatrix} 4 \\ 2 \\ 0 \end{pmatrix}, \mathbf{x}_3 = \begin{pmatrix} 3 \\ 3 \\ 1 \end{pmatrix}, \mathbf{x}_4 = \begin{pmatrix} 1 \\ 3 \\ 1 \end{pmatrix}, \mathbf{x}_5 = \begin{pmatrix} 3 \\ 1 \\ 0 \end{pmatrix}, \mathbf{x}_6 = \begin{pmatrix} 2 \\ 4 \\ 1 \end{pmatrix}$$

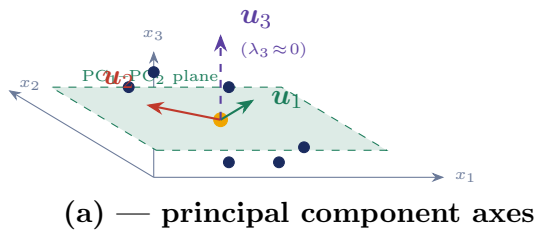
Mean: $\bar{\mathbf{x}} = \frac{1}{6}(15, 14, 3)^\top = (2.5, 2.33, 0.5)^\top$ (approximately).

Key outcomes after PCA:

- $\lambda_1 \gg \lambda_2 \gg \lambda_3 \approx 0$: the data is nearly flat (lies close to a 2D plane).
- PC1 ($\mathbf{u}_1$) points along the dominant spread direction (roughly the $(1, 0, 0)$ – $(0, 1, 0)$ plane).
- PC2 ($\mathbf{u}_2$) captures the secondary spread.
- PC3 ($\mathbf{u}_3$) has near-zero variance—the “thickness” perpendicular to the plane is negligible.
- We project to 2D: $\mathbf{y}_i = [\mathbf{u}_1 \mid \mathbf{u}_2]^\top (\mathbf{x}_i - \bar{\mathbf{x}}) \in \mathbb{R}^2$, discarding the near-zero PC3.



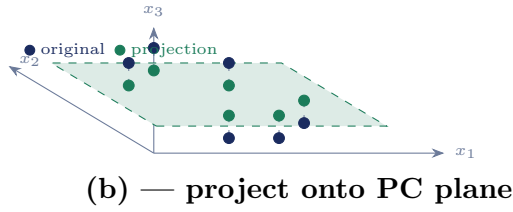
The raw data. Six points $\mathbf{x}_1, \dots, \mathbf{x}_6 \in \mathbb{R}^3$ plotted in 3D (cabinet oblique projection). They spread in the x_1 – x_2 plane but barely move in x_3 ($x_3 \in \{0, 1\}$), hinting that a 2D description may suffice. The amber dot is the sample mean $\bar{\mathbf{x}} = (2.5, 2.33, 0.5)^\top$.



(a) The eigenvectors of $\mathbf{S}$. After mean-centring and computing the covariance matrix $\mathbf{S}$, its eigenvectors are:

- $\mathbf{u}_1$ (PC1): direction of *greatest* variance, lying in the x_1 – x_2 plane (λ_1 large).
- $\mathbf{u}_2$ (PC2): orthogonal to $\mathbf{u}_1$, captures secondary spread (λ_2 moderate).
- $\mathbf{u}_3$ (PC3): perpendicular to the shaded plane; almost no variation here ($\lambda_3 \approx 0$, dashed).

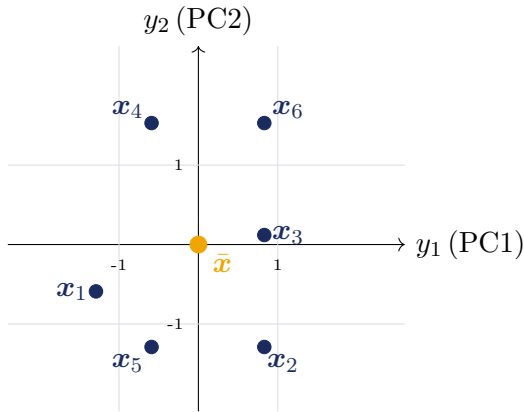
The shaded region is the PC_1 – PC_2 subspace we project onto.



(b) Orthogonal projection. Each point $\mathbf{x}_i$ is dropped *perpendicularly* onto the PC_1 – PC_2 plane (dashed lines). The green foot is

$$\tilde{\mathbf{x}}_i^\parallel = \mathbf{U}\mathbf{U}^\top(\mathbf{x}_i - \bar{\mathbf{x}}), \quad \mathbf{U} = [\mathbf{u}_1 | \mathbf{u}_2].$$

Points with $x_3 = 0$ are already near the plane (short drops); points with $x_3 = 1$ drop a little further. The discarded $\mathbf{u}_3$ captures only this small vertical spread.



(c) The 2D result. The projected coordinates are $\mathbf{y}_i = \mathbf{U}^\top(\mathbf{x}_i - \bar{\mathbf{x}}) \in \mathbb{R}^2$, with $y_1 = \mathbf{u}_1^\top \tilde{\mathbf{x}}_i$ and $y_2 = \mathbf{u}_2^\top \tilde{\mathbf{x}}_i$. The mean maps to the origin.

The six points spread clearly in this plane. Since $\lambda_3 \approx 0$, discarding $\mathbf{u}_3$ costs almost nothing: we go from 3D to 2D with *no perceptible information loss*.

What happened? The six 3D points lie near a flat plane (the $x_3 \approx \text{constant}$ surface). PCA detected this: $\lambda_3 \approx 0$, so the third component $\mathbf{u}_3$ is discarded. Panel (b) shows the 2D scatter in the PC_1 – PC_2 coordinate system, preserving all meaningful structure with no perceptible loss.